ACADEMIC PROJECT & VIVA DEFENSE GUIDE

CineGraph

Graph-Powered Movie Recommendation & Knowledge Discovery Platform using Neo4j, Express, and React

Project Title:	**CineGraph (Graph-Based Movie Recommendations)**
Database Technology:	Neo4j Graph Database (Cypher Query Language, Bolt Protocol)
Backend Framework:	Node.js & Express (RESTful MVC Architecture)
Frontend Stack:	React 18, Vite, Cytoscape.js, Recharts, Tailwind CSS
Key Algorithms:	Collaborative Filtering (Jaccard Index), Content-Based (Genre), Cast & Crew Centrality, Hybrid Ensemble

Prepared for Faculty Viva Voce & Laboratory Project Evaluation

1. Executive Summary & 30-Second Elevator Pitch

When presenting your project to your teacher or evaluation panel, begin with this confident opening:

> **"Good morning/afternoon, Ma'am:**
> *Our project is* **CineGraph**, *a full-stack movie exploration and recommendation platform powered by* **Neo4j**, **Express**, *and* **React**.
>
> *Unlike conventional platforms like IMDb or Netflix that rely on relational tables with costly multi-table SQL joins, CineGraph treats movies, genres, actors, directors, and users as an* **interconnected Knowledge Graph**.
>
> *Using native graph traversals, we implement* **4 distinct recommendation algorithms**—*Collaborative Filtering, Genre Affinity, Cast & Crew Connectivity, and a weighted Hybrid Ensemble—with* **transparent, explainable evidence** *for every suggestion, paired with an interactive* **Cytoscape.js** *visual graph explorer."*

2. Relational Database vs. Graph Database (The Core Problem)

Professors frequently ask: *"Why Neo4j instead of MySQL, PostgreSQL, or MongoDB?"*

Dimension	Relational Database (SQL)	Graph Database (Neo4j)
Data Storage	Tables with Rows and Columns. Relationships are simulated using Foreign Keys and intermediate join tables.	Nodes (entities), Properties (attributes), and Relationships (edges) stored as first-class citizens.
Relationship Traversal	**Cartesian JOINs:** Computes intersections by scanning foreign key indexes. Multi-hop queries cause *JOIN explosion*.	**Index-Free Adjacency:** Each node holds direct memory pointers to adjacent edges. Traversal is constant time $O(1)$ per hop.
Query Complexity	Exponential complexity $O(N^k)$ where N is table row count and k is traversal depth.	Sub-graph localized complexity $O(d^k)$ where d is average node degree. Total database size has zero impact.
Query Language	SQL (declarative tabular queries with nested subqueries).	Cypher (declarative pattern-matching language with visual ASCII art syntax).
Schema Flexibility	Rigid tabular schema; altering schema requires DDL migrations and table locks.	Flexible property graph model; new relationship types or properties can be added dynamically.

> **The "JOIN Explosion" Explanation (Say This to Ma'am):**
>
> "In SQL, finding movies liked by users who also loved films directed by the directors of movies I rated requires joining: `users → ratings → movie_directors → directors → movie_directors → movies → ratings → candidate_movies` . That requires **7 expensive joins**! In Neo4j, we simply traverse direct pointer references: `(u)-[:RATED]→(m)←[:DIRECTED]-(d)-[:DIRECTED]→(c)` in milliseconds."

3. CineGraph Data Model & Graph Schema

CineGraph models the cinematic domain as a labeled property graph containing **5 Node Labels** and **7 Relationship Types**:

Node Labels (Entities)

- `(:Movie)` : 80 curated titles. Properties: `movieId` , `title` , `releaseYear` , `rating` , `runtime` , `overview` , `addedAt` .
- `(:User)` : 8 seeded demo personas. Properties: `userId` , `name` , `email` , `passwordHash` .
- `(:Genre)` : 15 categories (Sci-Fi, Thriller, Drama, etc.). Properties: `genreId` , `name` .
- `(:Actor)` : 178 notable actors. Properties: `actorId` , `name` .
- `(:Director)` : 41 filmmakers. Properties: `directorId` , `name` .

Relationships (Edges & Semantics)

- `(User)-[:RATED {rating, ratedAt}]→(Movie)`
- `(User)-[:LIKED {likedAt}]→(Movie)`
- `(User)-[:WATCHED {watchedAt}]→(Movie)`
- `(User)-[:PREFERS]→(Genre)`
- `(Movie)-[:IN_GENRE]→(Genre)`
- `(Movie)-[:DIRECTED_BY]→(Director)`
- `(Movie)-[:ACTED_IN {role}]→(Actor)`

Graph Database Statistics in Current Database:

- **322 Total Nodes** (80 Movies + 15 Genres + 178 Actors + 41 Directors + 8 Users)
- **733 Total Relationships** connecting all nodes seamlessly
- Zero orphan nodes; all components form a cohesive connected knowledge network.

4. Schema Constraints & Range Indexes (Neo4j 5.x)

To enforce relational integrity and guarantee lightning-fast query execution, we defined explicit schema rules:

A. 7 Uniqueness Constraints:

```
// Neo4j 5 syntax: FOR (var:Label) REQUIRE var.property IS UNIQUE
CREATE CONSTRAINT user_id_unique IF NOT EXISTS FOR (u:User) REQUIRE u.userId IS UNIQUE;
CREATE CONSTRAINT user_email_unique IF NOT EXISTS FOR (u:User) REQUIRE u.email IS UNIQUE;
CREATE CONSTRAINT movie_id_unique IF NOT EXISTS FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
CREATE CONSTRAINT genre_id_unique IF NOT EXISTS FOR (g:Genre) REQUIRE g.genreId IS UNIQUE;
CREATE CONSTRAINT actor_id_unique IF NOT EXISTS FOR (a:Actor) REQUIRE a.actorId IS UNIQUE;
CREATE CONSTRAINT director_id_unique IF NOT EXISTS FOR (d:Director) REQUIRE d.directorId IS U
```

> **Why this matters to the examiner:**
> Uniqueness constraints guarantee that `MERGE` operations never create duplicate records. Furthermore, Neo4j automatically creates an underlying B-Tree index for each constraint, enabling $O(1)$ node lookups by primary identifier.

B. 6 Range Indexes for Search & Filtering:

```
CREATE INDEX movie_title IF NOT EXISTS FOR (m:Movie) ON (m.title);
CREATE INDEX movie_release_year IF NOT EXISTS FOR (m:Movie) ON (m.releaseYear);
CREATE INDEX movie_rating IF NOT EXISTS FOR (m:Movie) ON (m.rating);
CREATE INDEX actor_name IF NOT EXISTS FOR (a:Actor) ON (a.name);
CREATE INDEX director_name IF NOT EXISTS FOR (d:Director) ON (d.name);
```

5. The Recommendation Engine: Algorithms & Cypher Queries

CineGraph employs **4 core strategies** combined into an ensemble hybrid scoring model:

Strategy 1: Collaborative Filtering (Jaccard Index)

Discovers other users with overlapping taste and suggests unconsumed movies endorsed by them.

> **Mathematical Formulation:**
> Similarity between User A and User B is calculated using the **Jaccard Similarity Index** over their positive movie sets ($R \ge 4$ stars or liked): $$\text{Similarity}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

```
// Cypher for User-to-User Jaccard Similarity
UNWIND $seeds AS seed
MATCH (s:Movie {movieId: seed.movieId})←[r:LIKED|RATED]-(other:User)
WHERE other.userId <> $userId AND (type(r) = 'LIKED' OR r.rating ⩾ 4)
WITH other, collect(DISTINCT s) AS sharedMovies
WHERE size(sharedMovies) ⩾ $minSharedMovies

// Count other user's total positive library
MATCH (other)-[r2:LIKED|RATED]→(p:Movie)
WHERE type(r2) = 'LIKED' OR r2.rating ⩾ 4
WITH other, sharedMovies, count(DISTINCT p) AS otherPositiveCount

// Compute Jaccard coefficient
WITH other, sharedMovies,
     toFloat(size(sharedMovies)) / (size($seeds) + otherPositiveCount - size(sharedMovies)) A
RETURN other.userId, other.name, similarity, [x IN sharedMovies | x.title] AS sharedTitles
ORDER BY similarity DESC LIMIT 10;
```

Strategy 2: Content-Based Filtering (Genre Affinity)

Aggregates user taste weights across genres from high-rated movies and explicit onboarding preferences.

```
// Match unconsumed candidate movies sharing high-weight genres
UNWIND $genreWeights AS gw
MATCH (g:Genre {genreId: gw.genreId})←[:IN_GENRE]-(c:Movie)
WHERE NOT c.movieId IN $excludeIds
WITH c, collect(g.name) AS sharedGenres, sum(gw.weight) AS genreRaw
ORDER BY genreRaw DESC, c.rating DESC LIMIT 20;
```

Strategy 3: Cast & Crew Connectivity (Two-Hop Traversal)

Finds films sharing actors or directors that the user repeatedly praised in past ratings.

```
// Traversal: Seed Movie ⟵ Actor ⟶ Candidate Movie
UNWIND $seeds AS seed
MATCH (s:Movie {movieId: seed.movieId})←[:ACTED_IN]-(a:Actor)-[:ACTED_IN]→(c:Movie)
WHERE NOT c.movieId IN $excludeIds AND c.movieId <> s.movieId
WITH c, a, max(seed.weight) AS personWeight, collect(DISTINCT s.title) AS via
WITH c, collect({name: a.name, weight: personWeight, via: via}) AS people, sum(personWeight)
RETURN c.movieId, people, raw ORDER BY raw DESC LIMIT 20;
```

Strategy 4: The Hybrid Scoring Ensemble

Each independent signal is normalized between 0.0 and 1.0, and blended deterministically:

$$\text{Final Score} = w_{\text{genre}} \cdot S_{\text{genre}} + w_{\text{people}} \cdot S_{\text{people}} + w_{\text{collab}} \cdot S_{\text{collab}} + w_{\text{rating}} \cdot S_{\text{rating}}$$
Every recommendation produces an **Explanation Proof** indicating exactly which seed movie, genre, actor, or similar user triggered the suggestion.

6. ACID Compliance & Transaction Architecture

When asked how the database maintains consistency and avoids deadlocks:

How Neo4j Implements ACID:

- **Atomicity:** Every Cypher statement executes inside a transaction. If any constraint fails, the entire transaction rolls back.
- **Consistency:** Schema constraints guarantee unique IDs and emails. Broken relationships cannot exist without source/target nodes.
- **Isolation:** Uses Read-Committed isolation with Multi-Version Concurrency Control (MVCC). Writers lock modified nodes/edges; readers never block writers.
- **Durability:** Every committed change is written to an append-only Write-Ahead Transaction Log (WAL) before acknowledging success.

Managed Transactions in Node.js (`backend/src/config/neo4j.js`)

We distinguish between Read and Write transactions using the official Neo4j JavaScript Driver:

- `session.executeRead()` : Used for recommendation retrieval, catalog search, and graph visualization. Read queries can be routed to read-replicas in a cluster.
- `session.executeWrite()` : Used for rating movies, adding favorites, and seeding nodes. Automatically handles transient connection retry policies.

7. Live Viva Voce Step-by-Step Demonstration Script

Step	Screen / Page	What to Demonstrate & Say to Ma'am
1	**Home Page** `http://localhost:5173`	"Ma'am, here is the CineGraph landing interface. It dynamically loads featured movies, top-rated films, and trending titles directly from Neo4j."
2	**Explore Page** `/explore`	"This is our catalog explorer with debounced instant search and multi-facet filtering (Genre, Year, Minimum Rating). Notice the instant response time enabled by our Neo4j Range Indexes."
3	**Graph Explorer** `/graph`	**(Crucial Step!)** "This is our interactive Cytoscape.js knowledge graph. Each node represents an entity. Clicking on 'Inception' visually highlights connected nodes: Director (Christopher Nolan), Actors (Leonardo DiCaprio), and Genres (Sci-Fi, Thriller)."
4	**Login** `/login`	Log in using: `aarav@cinegraph.demo` / `Demo@1234`. "Aarav is our pre-configured persona who loves Christopher Nolan and Sci-Fi movies."
5	**Recommendations** `/recommendations`	"Here are the personalized recommendations for Aarav. Look at films like *'The Prestige'* and *'Blade Runner 2049'*. Clicking **'Why recommended?'** displays the transparent evidence: matching director, high genre overlap, and similar user endorsements."
6	**Analytics** `/analytics`	"Here we use Recharts to display graph aggregate analytics: rating distributions, top connected actors, and community trends calculated directly via Cypher aggregation functions."

8. Top 10 Rapid-Fire Viva Questions & Master Answers

Q1: What is Cypher, and how does it compare to SQL?

Answer: Cypher is Neo4j's declarative graph query language. While SQL relies on tabular algebra and complex nested JOINs, Cypher uses visual pattern matching (e.g. `(u)-[:RATED]→(m)`) optimized for traversing pointer structures.

Q2: What is the Cold Start problem, and how did you solve it?

Answer: A cold start occurs when a brand-new user has no interaction history, preventing collaborative filtering. We solve this by having new users select initial genre preferences during onboarding (`User-[:PREFERS]→Genre`) and defaulting to high-centrality, top-rated movies until they provide ratings.

Q3: What is the difference between `CREATE` and `MERGE` in Cypher?

Answer: `CREATE` unconditionally inserts a new node or relationship every time, which can create duplicates. `MERGE` checks if the pattern already exists; if found, it binds to it; if not found, it creates it (acting like an atomic `UPSERT`).

Q4: What protocol does your backend use to talk to Neo4j?

Answer: We use the official binary **Bolt Protocol** on port **7687**. Bolt is a connection-oriented, multiplexed binary protocol that avoids the overhead of JSON serialization over HTTP.

Q5: How do you prevent Cypher Injection attacks?

Answer: We never concatenate raw user input strings into Cypher queries. All inputs are passed as strictly typed **query parameters** (e.g., `$userId`, `$excludeIds`), ensuring inputs are parsed as literals, never as executable Cypher clauses.

Q6: How do you ensure already watched movies are not recommended?

Answer: We first aggregate all movies connected to the user via `[:RATED]`, `[:LIKED]`, or `[:WATCHED]` into an `$excludeIds` array, and enforce `WHERE NOT candidate.movieId IN $excludeIds` in every recommendation query.

Q7: What is Degree Centrality in the context of your graph?

Answer: Degree Centrality measures the number of relationships incident to a node. High-degree actors (e.g. Leonardo DiCaprio) and directors act as key topological bridges in the graph connecting otherwise distant movie clusters.

Q8: What security practices did you implement on the web tier?

Answer: We implemented: (1) `bcryptjs` salted password hashing, (2) JWT tokens stored in `HttpOnly`, `SameSite=lax` cookies to eliminate XSS token theft, (3) Helmet security headers, (4) Express rate limiters on authentication routes, and (5) schema validation with `express-validator`.